# Grounded by Design: Building and Stress-Testing a Domain-Specific RAG Chatbot for Monster Hunter Lore

Luke Joaquin Bernardo
College of Computer Studies and Engineering
Jose Rizal University
Taguig City, Philippines
lukejoaquin.bernardo@my.jru.edu

***Abstract*—I built a chatbot that only answers questions about Monster Hunter: World and its Iceborne expansion, using retrieval-augmented generation (RAG) instead of relying on the model's own memory. The pipeline loads three lore files, splits them into small chunks, embeds them with all-MiniLM-L6-v2, and stores them in an in-memory ChromaDB index. A Groq-hosted language model then answers using only the chunks it retrieves. I tested this two ways: once with a strict system prompt and the temperature set to zero, and once with the refusal rule deleted and the temperature raised to 1.0. In both cases, the chatbot stayed correctly grounded on an out-of-domain question instead of making something up. What changed was the wording of its refusal, which became less consistent once the explicit fallback instruction was removed. That gap between staying factually grounded and staying predictably worded turned out to be the most useful thing I took from this lab.**

## I. System Architecture

The chatbot follows a standard retrieval-augmented generation pipeline, and I kept it close to the lab's provided template. It has five stages, and each one solves a different problem.

The first stage is document loading. I used LangChain's DirectoryLoader to read three plain text files from a local folder: one covering monster lore, one covering location and ecosystem lore, and one covering story and faction lore. All three came from base-game and Iceborne content, so the chatbot has a clear, narrow domain instead of trying to cover the whole Monster Hunter franchise.

The second stage is chunking. A single lore file is too big to hand to a retriever as one block, so I split each document into pieces of about 500 characters, with 50 characters of overlap between consecutive chunks. The overlap matters because it stops a sentence from getting cut in half and losing its meaning in both halves. Smaller chunks also let the retriever pull out just the part of a document that actually matches a question, instead of returning an entire file.

The third stage is embedding. Each chunk gets converted into a vector using all-MiniLM-L6-v2, a small open-source sentence embedding model. This turns text into a list of numbers that captures what it means, not just what words it contains, which is what makes semantic search possible in the next step.

The fourth stage is the vector store. I used ChromaDB, running in memory for this exercise, to hold all the chunk embeddings and let me search them by similarity. When a query comes in, ChromaDB returns the three chunks whose embeddings are closest to the query's embedding. Three was the value set by the lab template, and I kept it as-is.

The fifth stage is generation. The three retrieved chunks get stuffed into a prompt template alongside the user's question, and the whole thing goes to a Groq-hosted language model. The system prompt tells the model to answer only from the given context, and, in the baseline setup, to give a fixed refusal line if the answer isn't there. LangChain's `create_retrieval_chain` function wires the retriever and the language model together into one callable object, so a single function call handles retrieval and generation in sequence.

## II. Implementation Details

Table I lists the parameters I used for the baseline run and the parameters I changed for the hallucination stress-test in Section III.

TABLE I
Pipeline Parameters

| Parameter | Value |
|---|---|
| Chunk size | 500 characters |
| Chunk overlap | 50 characters |
| Retrieval top-k | 3 |
| Embedding model | all-MiniLM-L6-v2 |
| Vector store | ChromaDB (in-memory) |
| Language model | openai/gpt-oss-20b (Groq) |
| Temperature (baseline) | 0 |
| Temperature (stress-test) | 1.0 |

One implementation detail worth mentioning is the language model itself. The lab template originally specified llama-3.1-8b-instant, but Groq had deprecated that model by the time I ran this lab, so I swapped in openai/gpt-oss-20b instead, since it was the closest available replacement in size and purpose. This turned out to be a small, real-world example of something the lecture material covered directly: model deprecation and the need for version tracking are part of why LLMOps exists as its own discipline, separate from general MLOps. A pipeline

built around a hardcoded model name is fragile in exactly this way, and I only noticed it because the notebook broke.

I also want to be upfront about Cell 5's API key handling. The notebook uses Python's getpass module to collect the Groq API key at runtime instead of hardcoding it into a cell, so the key never gets saved into the notebook file itself. This is a small thing, but it directly reflects the basic API-key protection practices this lab asks us to reflect on in the conclusion.

Beyond the notebook, I also rebuilt this pipeline as a standalone Gradio application deployed on Hugging Face Spaces, which made it easy to switch between the baseline and stress-test configurations for the experiment in Section III.

## III. Experimental Results and Hallucination Analysis

Table II summarizes the four test queries I ran against the chatbot, what I expected each one to return, and what it actually returned.

TABLE II
Test Queries and Observed Outputs

| Test | Expected | Actual |
|---|---|---|
| In-domain, vague summary query | General summary | Grounded but narrow; pulled a trap-effectiveness table instead of a broad summary |
| In-domain, specific query (Nergigante) | Detailed, grounded answer | Correct, detailed, fully sourced from the monster lore file |
| Out-of-domain, baseline (temp=0, fallback on) | Refusal | Refused with the exact fallback line |
| Out-of-domain, stress-test (temp=1.0, fallback removed) | Uncertain | Still refused, but with looser, less consistent wording |

The vague in-domain query is a good example of a retrieval limitation rather than a chatbot failure. A question like "summarize the key information" doesn't closely match any single chunk, so the retriever just returned whichever three chunks scored highest by similarity, which happened to be about trap effectiveness. The model then faithfully summarized those chunks and correctly listed its source file, so the grounding itself worked. The mismatch was between the query and the retrieval, not between the answer and the retrieved context. The specific query about Nergigante shows what the same pipeline looks like when the query and the content actually line up, and the answer came back detailed and clearly sourced.

The baseline out-of-domain test worked exactly as intended. Asking about Magnamalo, a monster from a different game in the series that isn't covered by any of the three lore files, triggered the fixed refusal line word for word: it cannot answer based on the provided domain data.

The stress-test is where things got more interesting than I expected. I raised the temperature to 1.0 and deleted the line in the system prompt that gave the model its exact refusal wording, leaving only the instruction to answer strictly from the provided context. I expected this to either cause the model to hallucinate Magnamalo lore by pulling from its own training data, or to keep refusing as before. What happened was a third option: it still refused, but with looser, more conversational wording instead of the fixed sentence. It said it didn't have enough information in the provided context, rather than reciting the original refusal line.

This matters because it separates two things that are easy to lump together. One is grounding, meaning whether the model's answer actually stays tied to the retrieved chunks. The other is refusal consistency, meaning whether the model always phrases a refusal the same way. Removing the explicit fallback sentence did not break grounding in this run, since the surviving instruction to use only the provided context still did real work. But it did break consistency, and for a chatbot meant to be deployed, consistent refusal wording is its own requirement, separate from whether the model happens to stay honest on a given run. A system prompt that only tells a model what to do, without also fixing how to say no, is leaving that part up to chance.

## IV. Conclusion

Working through this lab changed how I think about what actually keeps a chatbot safe. Before this, I would have assumed the main risk was the model inventing facts. What I found instead is that the retrieved context and the system prompt do most of the real work, and the model's own judgment is a second layer, not the first one. Prompt restrictions matter because they set the ground rules before the model even sees a question, and RAG matters because it gives the model something real to point to instead of just its training data. Losing the explicit refusal line didn't cause a hallucination in my case, but it also isn't something I would want to rely on for every query in production, since one lucky result doesn't guarantee the next one goes the same way.

The API-key handling in this lab was a small detail, but it stood out to me as the same idea applied to a different problem. Keeping the key out of the notebook doesn't make the model smarter, but it stops a leaked credential from turning into a bigger failure. Both the prompt restrictions and the key handling come down to the same thing: not trusting any single part of the system to behave correctly on its own, and building in a check instead.

## References


[1] R. C. Raga Jr., "Introduction to LLM Ops and RAG Concepts," ITC C508 Lecture 4 slides, National University.

[2] LangChain, "LangChain v1 migration guide," https://docs.langchain.com/oss/python/migrate/langchain-v1.

[3] LangChain, "create_retrieval_chain — langchain_classic reference," https://reference.langchain.com/python/langchain-classic/chains/retrieval/create_retrieval_chain.

[4] Groq, "Supported Models," GroqCloud documentation, https://console.groq.com/docs/models.

[5] Groq, "Deprecations," GroqCloud documentation, https://console.groq.com/docs/deprecations.

[6] Sentence-Transformers, "all-MiniLM-L6-v2," Hugging Face model card, https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2.

[7] Chroma, "Chroma Documentation," https://docs.trychroma.com/.

[8] Monster Hunter Wiki contributors, Monster Hunter Wiki, Fandom, https://monsterhunter.fandom.com/wiki/Monster_Hunter_Wiki.